

DATA 622: Machine Learning & Big Data

Lab 1: Bias Variance Trade-Off Using kNN

Overview of the kNN

Before jumping into the code for Problem 1, here is a quick refresher on how the model actually makes its predictions. k-Nearest Neighbors (kNN) works on the simple premise that similar things exist close to each other. When the algorithm needs to classify a brand new piece of data, it doesn't rely on a complex internal equation. Instead, it just looks at the data points immediately surrounding it and takes a vote.

The k stands for the number of neighbors you want to check. If you set $k=5$, the algorithm looks at the 5 closest data points. If three of those neighbors are classified as cats and two are dogs, it labels the new data point as a cat.

Setup & Imports

Loading the tools needed to process the data, train the kNN models, and generate the plots.

```
In [2]: import os
import urllib.request
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, log_loss
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# Plotting configurations
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'seaborn-v0_8-dark')
plt.rcParams['figure.dpi'] = 120
plt.rcParams['font.size'] = 11
```

1. Problem 1: kNN Classification of the ZIP Code Digit Data

(a) Data Loading & Initial Inspection

- Download and load `zip.train` and `zip.test`.
- There is no header
- Zeroth column: class label (digits 0–9).
- Columns 1–256: pixel grayscale intensities normalized from 0 to 1.
- Select the first entry, reshape to 16 × 16, and plot the image.

```
In [10]: # URLs for the datasets
TRAIN_URL = "https://raw.githubusercontent.com/georgehagstrom/DATA622Fall12026/refs/
TEST_URL = "https://raw.githubusercontent.com/georgehagstrom/DATA622Fall12026/refs/h

print("Pulling zip.train directly from GitHub...")
train_df = pd.read_csv(TRAIN_URL, sep=r'\s+', header=None)

print("Pulling zip.test directly from GitHub...")
test_df = pd.read_csv(TEST_URL, sep=r'\s+', header=None)

# Extract labels and image features

zip_class_train = train_df.pop(0).astype(int)
zip_image_train = train_df.to_numpy()

zip_class_test = test_df.pop(0).astype(int)
zip_image_test = test_df.to_numpy()

# Condensed, standard output formatting
print(f"Train shape: {zip_image_train.shape} | Test shape: {zip_image_test.shape}")
print(f"Classes: {sorted(zip_class_train.unique())}")
```

Pulling zip.train directly from GitHub...

Pulling zip.test directly from GitHub...

Train shape: (7291, 256) | Test shape: (2007, 256)

Classes: [np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6), np.int64(7), np.int64(8), np.int64(9)]

- `pd.read_csv()`: This Pandas function reads the dataset directly from the GitHub URL.
 - The `sep=r'\s+'` argument is a regular expression that tells Pandas the columns are separated by any amount of whitespace (spaces or tabs), rather than standard commas.
 - The `header=None` argument prevents Pandas from accidentally turning your first row of actual image data into column names.
- `pop(0)`: Extracts and removes column 0 (the labels) from the dataframe. This leaves the original dataframe containing ONLY the image pixels.
- `astype(int)`: This forces the extracted labels to be integers.
- `to_numpy()`: Strips away the pandas row/column formatting and converts the data into a raw matrix, which is the exact format scikit-learn models require.

Plot the First Entry

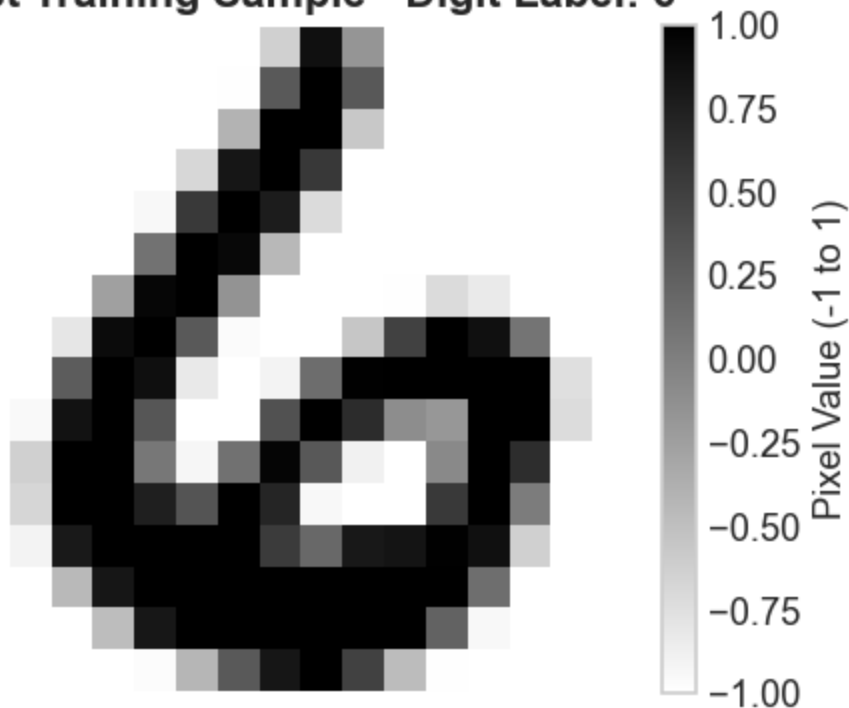
The request is to select the first entry in the training set, reshape it to 16×16 , and plot the image. To do that, we used the following functions to build our plot:

- `iloc[0]`: This pulls the very first label (row 0) from our target array so we can accurately display the correct digit in the image title. (Note: Because we already removed column 0 earlier, this grabs the first row's answer, not the column).
- `zip_image_train[0]` grabs the corresponding row of 256 pixel values
- `reshape((16, 16))`: This changes the flat matrix of 256 image features back into a 16×16 grid.
- `imshow()`: This renders the matrix visually on the screen using the matplotlib plot configuration we previously defined, applying a grayscale colormap to make it look like natural handwriting.

```
In [15]: # Plot the first entry in the training set
first_digit_label = zip_class_train.iloc[0] #row 0 Label
print(f"First training sample label: {first_digit_label}")
first_digit_image = zip_image_train[0].reshape((16, 16)) #row 0 image reshaped to 16x16
fig, ax = plt.subplots(figsize=(4, 4))
im = ax.imshow(first_digit_image, cmap='gray_r')
ax.set_title(f"First Training Sample - Digit Label: {first_digit_label}", fontsize=16)
ax.axis('off')
plt.colorbar(im, ax=ax, fraction=0.046, pad=0.04, label='Pixel Value (-1 to 1)')
plt.tight_layout()
plt.show()
```

First training sample label: 6

First Training Sample - Digit Label: 6



(b) Validation Curves (Accuracy vs. \$1/k\$)

The following code imports the kNN classification function from scikit-learn as well as an accuracy function, trains a kNN classifier, and tests its accuracy on hypothetical training and testing data:

```
In [18]: from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

knn = KNeighborsClassifier(n_neighbors=4)
knn.fit(zip_image_train, zip_class_train)

zip_pred_train = knn.predict(zip_image_train)
accuracy = accuracy_score(zip_class_train, zip_pred_train)
print(f'Accuracy: {accuracy:.4f}')

zip_pred_test = knn.predict(zip_image_test)
accuracy = accuracy_score(zip_class_test, zip_pred_test)
print(f'Accuracy: {accuracy:.4f}')

train_correct_predictions = (zip_pred_train == zip_class_train.to_numpy()).sum()
test_correct_predictions = (zip_pred_test == zip_class_test.to_numpy()).sum()
print(f'Correct training predictions: {train_correct_predictions}')
print(f'Correct test predictions: {test_correct_predictions}')
```

Accuracy: 0.9807

Accuracy: 0.9432

Correct training predictions: 7150

Correct test predictions: 1893

Interpretation of the Accuracy Results

With $k=4$, the kNN classifier correctly labels **98.07%** of the training images and **94.32%** of the separate test images. Each accuracy is calculated as $\frac{\text{number of correct predictions}}{\text{total number of images}}$.

Dataset	Total images	Correct predictions	Accuracy
Training	7,291	7,150	$\$7150 / 7291 = 0.9807\$$ (98.07%)
Test	2,007	1,893	$\$1893 / 2007 = 0.9432\$$ (94.32%)

The error rate is the proportion of predictions that are incorrect. It can be calculated either as $\frac{\text{number of incorrect predictions}}{\text{total number of images}}$ or as $1 - \text{accuracy}$.

Dataset	Total images	Incorrect predictions	Error rate
Training	7,291	141	$\$141 / 7291 = 0.0193\$$ (1.93%)

Dataset	Total images	Incorrect predictions	Error rate
Test	2,007	114	$114 / 2007 = 0.0568$ (5.68%)

The lower test accuracy is expected because the model was fitted on the training data. The difference of $0.9807 - 0.9432 = 0.0375$, or **3.75 percentage points**, indicates some overfitting: the model fits the examples it has seen slightly better than new handwritten digits. However, the test result is still high, so the model generalizes well overall. The test accuracy is the more useful estimate of how accurately this $k=4$ model will classify future unseen digit images.

Recreate **Figure 2.17 from Introduction to Statistical Learning (ISLR)**:

- x-axis: Flexibility ($1/k$), with k ranging from 1 to 300.
- y-axis: Error Rate (or Accuracy) for both Training and Testing sets.
- Analyze the U-shaped testing error curve and the divergence from training error.

```
In [19]: # Define range of k values
k_values = [1, 2, 3, 4, 5, 7, 9, 11, 15, 21, 31, 45, 65, 95, 140, 200, 300]

train_acc = []
test_acc = []
train_err = []
test_err = []

print("Training kNN models across k in [1, 300]...")
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k, n_jobs=-1)
    knn.fit(zip_image_train, zip_class_train)

    pred_train = knn.predict(zip_image_train)
    pred_test = knn.predict(zip_image_test)

    acc_tr = accuracy_score(zip_class_train, pred_train)
    acc_te = accuracy_score(zip_class_test, pred_test)

    train_acc.append(acc_tr)
    test_acc.append(acc_te)
    train_err.append(1.0 - acc_tr)
    test_err.append(1.0 - acc_te)
    print(f"k = {k:3d} | 1/k = {1/k:0.4f} | Train Acc = {acc_tr:.4f} (Err: {1-acc_t

inv_k = [1.0 / k for k in k_values]
```

Training kNN models across k in [1, 300]...

k	1/k	Train Acc	Train Err	Test Acc	Test Err
1	1.0000	1.0000	0.0000	0.9437	0.0563
2	0.5000	0.9837	0.0163	0.9412	0.0588
3	0.3333	0.9867	0.0133	0.9447	0.0553
4	0.2500	0.9807	0.0193	0.9432	0.0568
5	0.2000	0.9792	0.0208	0.9447	0.0553
7	0.1429	0.9742	0.0258	0.9417	0.0583
9	0.1111	0.9709	0.0291	0.9372	0.0628
11	0.0909	0.9690	0.0310	0.9312	0.0688
15	0.0667	0.9628	0.0372	0.9302	0.0698
21	0.0476	0.9521	0.0479	0.9193	0.0807
31	0.0323	0.9424	0.0576	0.9103	0.0897
45	0.0222	0.9333	0.0667	0.8999	0.1001
65	0.0154	0.9204	0.0796	0.8844	0.1156
95	0.0105	0.9054	0.0946	0.8675	0.1325
140	0.0071	0.8849	0.1151	0.8480	0.1520
200	0.0050	0.8622	0.1378	0.8281	0.1719
300	0.0033	0.8360	0.1640	0.7997	0.2003

Plot Validation Curves

```
In [20]: # Plot Validation Curves (ISLR Figure 2.17 style)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 1. Error Rate vs 1/k
axes[0].plot(inv_k, train_err, marker='o', label='Training Error', color='tab:blue')
axes[0].plot(inv_k, test_err, marker='s', label='Test Error', color='tab:orange')
axes[0].set_xlabel('Flexibility ($1/k$)', fontsize=12, weight='bold')
axes[0].set_ylabel('Error Rate', fontsize=12, weight='bold')
axes[0].set_title('Classification Error vs. $1/k$ (ISLR Fig 2.17)', fontsize=13, weight='bold')
axes[0].legend(frameon=True)
axes[0].grid(True, linestyle='--', alpha=0.6)

# 2. Accuracy vs k
axes[1].plot(k_values, train_acc, marker='o', label='Training Accuracy', color='tab:blue')
axes[1].plot(k_values, test_acc, marker='s', label='Test Accuracy', color='tab:orange')
axes[1].set_xlabel('Number of Neighbors ($k$)', fontsize=12, weight='bold')
axes[1].set_ylabel('Accuracy', fontsize=12, weight='bold')
axes[1].set_title('Accuracy vs. $k$ (ISLR Fig 2.17)', fontsize=13, weight='bold')
```

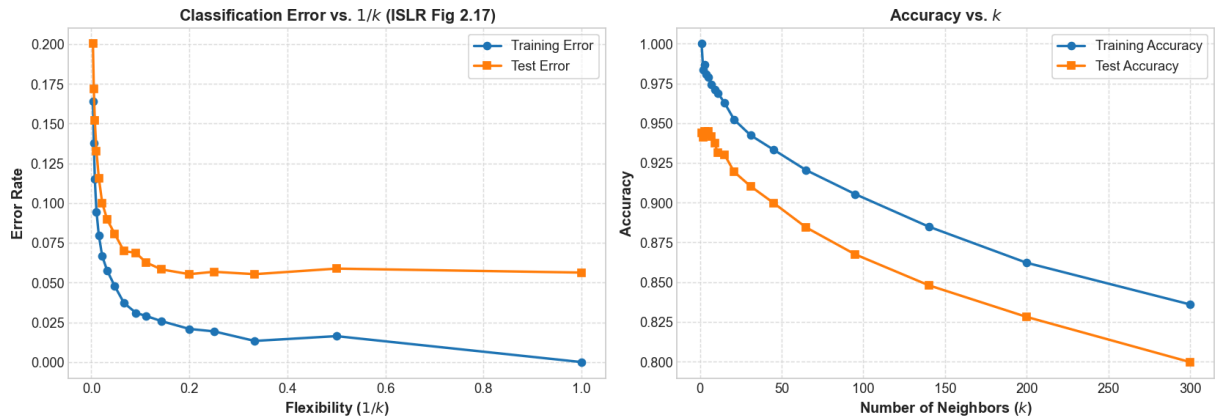
```

axes[1].legend(frameon=True)
axes[1].grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()

best_k_idx = np.argmin(test_err)
print(f"Optimal k for Accuracy: k = {k_values[best_k_idx]} with Test Accuracy = {te

```



Optimal k for Accuracy: k = 3 with Test Accuracy = 0.9447 (Test Error = 0.0553)

Plot Interpretation

These curves show how the kNN classifier performs when k changes. Small values of k use only a few nearby images for each prediction. Large values of k use more neighbors and produce smoother decisions.

The smallest test error occurs at both $k=3$ and $k=5$. Each gives **94.47%** accuracy and **5.53%** error on the test set. The code prints $k=3$ because `np.argmin(test_err)` returns the first value with the minimum error. The two settings give the same result on this test set, so choosing $k=3$ is reasonable because it uses fewer neighbors.

At $k=1$, training accuracy is **100%** because each training image is its own nearest neighbor. Test accuracy is **94.37%**, which is lower than the result for $k=3$ and $k=5$. This difference is a sign of overfitting: the model matches the training images extremely closely but does not perform quite as well on unseen images.

When k increases from 1 to 3 , the test error falls because a few neighbors reduce the effect of unusual individual images. With much larger values of k , the classifier averages too many neighbors. This can blur the boundaries between digit classes, increase bias, and lower accuracy. The best values of k balance these two effects.

(c) Log Loss Validation Curves

Log loss is a grading system that evaluates a machine learning model based on the confidence of its predictions, rather than just whether it was right or wrong. While standard

accuracy only checks the final answer, the purpose of log loss is to measure how well the model estimates the probability of a sample belonging to each class.

Log loss penalizes overconfident wrong predictions much more heavily than accuracy. The purpose of log loss is to estimate the probability of an unknown sample being in each class, rather than to predict just the most likely class. You can apply log loss using the `log_loss` function built into `sklearn` as follows

```
In [21]: from sklearn.metrics import log_loss

knn = KNeighborsClassifier(n_neighbors=4)
knn.fit(zip_image_train, zip_class_train)

zip_prob_train = knn.predict_proba(zip_image_train) # Notice how we are predicting
loss_train = log_loss(zip_class_train, zip_prob_train, labels = knn.classes_)
print(f'loss_train: {loss_train:.4f}')

zip_prob_test = knn.predict_proba(zip_image_test)
loss_test = log_loss(zip_class_test, zip_prob_test, labels = knn.classes_)
print(f'loss_test: {loss_test:.4f}')
```

```
loss_train: 0.0420
```

```
loss_test: 1.1045
```

Repeat exercise (b) using `log_loss` instead of accuracy. Compare and contrast the two validation curves you generated and the optimal values of observed in each. It is said in Machine Learning that probability estimation is more difficult than classification. How does that manifest with the values of observed in (b) and (c)?

In order to resolve that we will do the following steps:

- Evaluate model calibration using `predict_proba()` and `log_loss()`.
- Compare and contrast accuracy vs. log loss validation curves and optimal k values.
- Discuss why probability estimation is harder than pure classification.

```
In [22]: train_logloss = []
test_logloss = []

print("Computing Log Loss across k values...")
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k, n_jobs=-1)
    knn.fit(zip_image_train, zip_class_train)

    prob_tr = knn.predict_proba(zip_image_train)
    prob_te = knn.predict_proba(zip_image_test)

    loss_tr = log_loss(zip_class_train, prob_tr, labels=knn.classes_)
    loss_te = log_loss(zip_class_test, prob_te, labels=knn.classes_)

    train_logloss.append(loss_tr)
    test_logloss.append(loss_te)
    print(f"k = {k:3d} | Train Log Loss: {loss_tr:.4f} | Test Log Loss: {loss_te:.4f}")
```

Computing Log Loss across k values...

k = 1	Train Log Loss: 0.0000	Test Log Loss: 2.0294
k = 2	Train Log Loss: 0.0194	Test Log Loss: 1.4671
k = 3	Train Log Loss: 0.0312	Test Log Loss: 1.2674
k = 4	Train Log Loss: 0.0420	Test Log Loss: 1.1045
k = 5	Train Log Loss: 0.0507	Test Log Loss: 1.0424
k = 7	Train Log Loss: 0.0665	Test Log Loss: 0.9531
k = 9	Train Log Loss: 0.0790	Test Log Loss: 0.8618
k = 11	Train Log Loss: 0.0896	Test Log Loss: 0.7351
k = 15	Train Log Loss: 0.1085	Test Log Loss: 0.7017
k = 21	Train Log Loss: 0.1314	Test Log Loss: 0.6226
k = 31	Train Log Loss: 0.1626	Test Log Loss: 0.5219
k = 45	Train Log Loss: 0.1973	Test Log Loss: 0.5078
k = 65	Train Log Loss: 0.2373	Test Log Loss: 0.4828
k = 95	Train Log Loss: 0.2872	Test Log Loss: 0.5156
k = 140	Train Log Loss: 0.3488	Test Log Loss: 0.5789
k = 200	Train Log Loss: 0.4166	Test Log Loss: 0.6293
k = 300	Train Log Loss: 0.5109	Test Log Loss: 0.6878

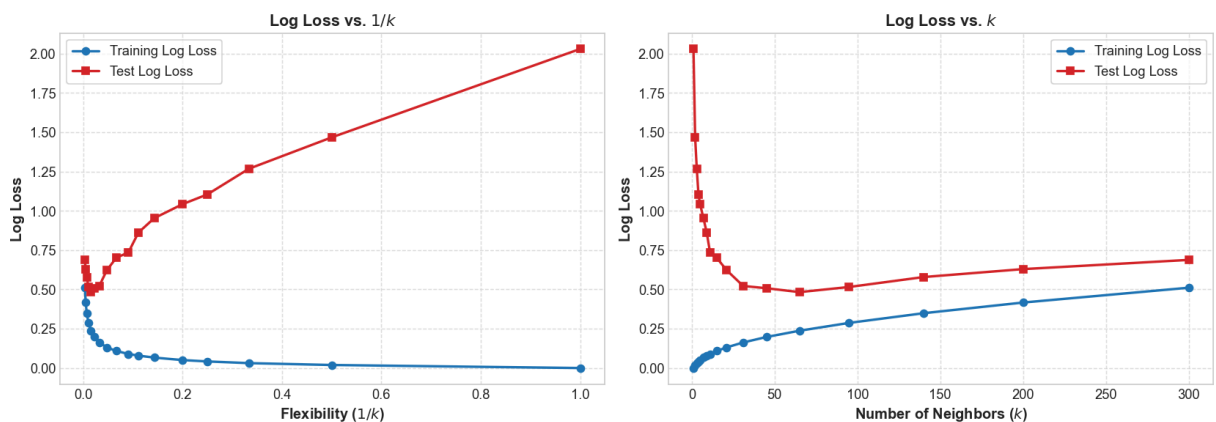
```
In [7]: # Plot Log Loss vs 1/k and k
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(inv_k, train_logloss, marker='o', label='Training Log Loss', color='tab:blue')
axes[0].plot(inv_k, test_logloss, marker='s', label='Test Log Loss', color='tab:red')
axes[0].set_xlabel('Flexibility ($1/k$)', fontsize=12, weight='bold')
axes[0].set_ylabel('Log Loss', fontsize=12, weight='bold')
axes[0].set_title('Log Loss vs. $1/k$', fontsize=13, weight='bold')
axes[0].legend(frameon=True)
axes[0].grid(True, linestyle='--', alpha=0.6)

axes[1].plot(k_values, train_logloss, marker='o', label='Training Log Loss', color='tab:blue')
axes[1].plot(k_values, test_logloss, marker='s', label='Test Log Loss', color='tab:red')
axes[1].set_xlabel('Number of Neighbors ($k$)', fontsize=12, weight='bold')
axes[1].set_ylabel('Log Loss', fontsize=12, weight='bold')
axes[1].set_title('Log Loss vs. $k$', fontsize=13, weight='bold')
axes[1].legend(frameon=True)
axes[1].grid(True, linestyle='--', alpha=0.6)

plt.tight_layout()
plt.show()

best_k_ll_idx = np.argmin(test_logloss)
print(f"Optimal k for Log Loss: k = {k_values[best_k_ll_idx]} with Test Log Loss =
```



Optimal k for Log Loss: $k = 65$ with Test Log Loss = 0.4828

Plot Interpretation - Log Loss

Left chart: Log Loss vs. $1/k$. The horizontal axis shows flexibility as $1/k$. Moving right means $1/k$ gets larger, so k gets smaller and the classifier uses fewer neighbors. The blue line shows training log loss, and the red line shows log loss on the test data. The lowest point on the red line gives the most reliable probability estimates for unseen images.

The value $1/k$ changes in the opposite direction from k :

k neighbors	$1/k$	Flexibility
300	0.0033	Low: averages many neighbors
65	0.0154	Low; gives the smallest test log loss
10	0.1000	Moderate
5	0.2000	High
1	1.0000	Highest: uses one neighbor

For example, when k drops from 10 to 1, $1/k$ rises from 0.10 to 1.00. Using fewer neighbors makes each prediction depend more on individual training images.

Right chart: Log Loss vs. k . This chart shows the same values with k on the horizontal axis. Moving right means the classifier uses more neighbors for each prediction. The lowest point of the red line occurs at $k=65$.

For this data, $k=65$ gives the smallest test log loss, **0.4828**. A smaller log loss is better because it gives a lower value when the model assigns more probability to the correct digit. It gives a larger value when the model is confident about the wrong digit.

This differs from the accuracy result, where $k=3$ and $k=5$ are best. Accuracy only asks whether the predicted class is correct. Log loss also considers the probability given to the correct class, so it favors less extreme probability estimates.

2. Problem 2: Learning Curves

(a) Learning Curves on ZIP Code Data

- Subsample the training dataset across varying sample sizes N .
- For each sample size, find the optimal k that minimizes test error.
- Plot training error, test error, and optimal k vs. sample size (both linear and log-log scale).

- Discuss whether test error has plateaued and evaluate strategic options (more data vs. complex models vs. feature engineering).

```
In [23]: def find_best_k_for_subsample(X_sub, y_sub, X_val, y_val, candidate_k=[1, 3, 5, 7,
    """Finds the k with the highest accuracy / lowest test error for a given traini
    best_k = candidate_k[0]
    best_err = float('inf')
    best_train_err = 0.0

    for k in candidate_k:
        if k > len(y_sub):
            continue
        knn = KNeighborsClassifier(n_neighbors=k, n_jobs=-1)
        knn.fit(X_sub, y_sub)

        te_acc = accuracy_score(y_val, knn.predict(X_val))
        te_err = 1.0 - te_acc

        if te_err < best_err:
            best_err = te_err
            best_k = k
            tr_acc = accuracy_score(y_sub, knn.predict(X_sub))
            best_train_err = 1.0 - tr_acc

    return best_k, best_train_err, best_err
```

find_best_k_for_subsample

This function chooses a value of k for one subset of the training data. The notebook calls it once for each training sample size so it can build the learning curves.

Input	Meaning
<code>X_sub , y_sub</code>	Images and labels in the current training subset
<code>X_val , y_val</code>	Images and labels used to evaluate each value of k
<code>candidate_k</code>	Values of k to try, such as $1, 3, 5, \dots$

The function trains one kNN model for every value in `candidate_k`. It predicts the labels in `X_val`, calculates accuracy, and then calculates error with $\text{error} = 1 - \text{accuracy}$. It saves the value of k that gives the smallest error.

Returned value	Meaning
<code>best_k</code>	The value of k with the smallest error on the evaluation data
<code>best_train_err</code>	Training error when the model uses <code>best_k</code>
<code>best_err</code>	Smallest error found on the evaluation data

`if k > len(y_sub): continue` skips values that are larger than the number of images in the subset. A kNN model cannot use more neighbors than there are training images.

```

In [30]: sample_sizes = [100, 250, 500, 1000, 2000, 4000, len(zip_image_train)]
lc_results = []

np.random.seed(42)
for n in sample_sizes:
    if n == len(zip_image_train):
        X_sub, y_sub = zip_image_train, zip_class_train
    else:
        idx = np.random.choice(len(zip_image_train), size=n, replace=False)
        X_sub = zip_image_train[idx]
        y_sub = zip_class_train.iloc[idx]

    best_k, tr_err, te_err = find_best_k_for_subsample(
        X_sub, y_sub, zip_image_test, zip_class_test,
        candidate_k=[1, 3, 5, 7, 9, 15, 21, 31, 45]
    )
    lc_results.append({
        'sample_size': n,
        'best_k': best_k,
        'train_error': tr_err,
        'test_error': te_err,
        'test_accuracy': 1.0 - te_err
    })
    print(f"N = {n:5d} | Optimal k = {best_k:2d} | Train Err = {tr_err:.4f} | Test

lc_df = pd.DataFrame(lc_results)

```

```

N =   100 | Optimal k =  1 | Train Err = 0.0000 | Test Err = 0.2566
N =   250 | Optimal k =  1 | Train Err = 0.0000 | Test Err = 0.1779
N =   500 | Optimal k =  1 | Train Err = 0.0000 | Test Err = 0.1355
N =  1000 | Optimal k =  1 | Train Err = 0.0000 | Test Err = 0.0982
N =  2000 | Optimal k =  1 | Train Err = 0.0000 | Test Err = 0.0847
N =  4000 | Optimal k =  1 | Train Err = 0.0000 | Test Err = 0.0648
N =  7291 | Optimal k =  3 | Train Err = 0.0133 | Test Err = 0.0553

```

```

In [31]: # Plot Learning Curves (Linear and Log-Log scale)
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. Classification error on a Linear scale
axes[0, 0].plot(lc_df['sample_size'], lc_df['train_error'], marker='o', label='Train Error')
axes[0, 0].plot(lc_df['sample_size'], lc_df['test_error'], marker='s', label='Test Error')
axes[0, 0].set_xlabel('Training Sample Size (N)', fontsize=12, weight='bold')
axes[0, 0].set_ylabel('Error Rate', fontsize=12, weight='bold')
axes[0, 0].set_title('Learning Curve: ZIP Code Data (Linear Scale)', fontsize=13, weight='bold')
axes[0, 0].legend(frameon=True)
axes[0, 0].grid(True, linestyle='--', alpha=0.6)

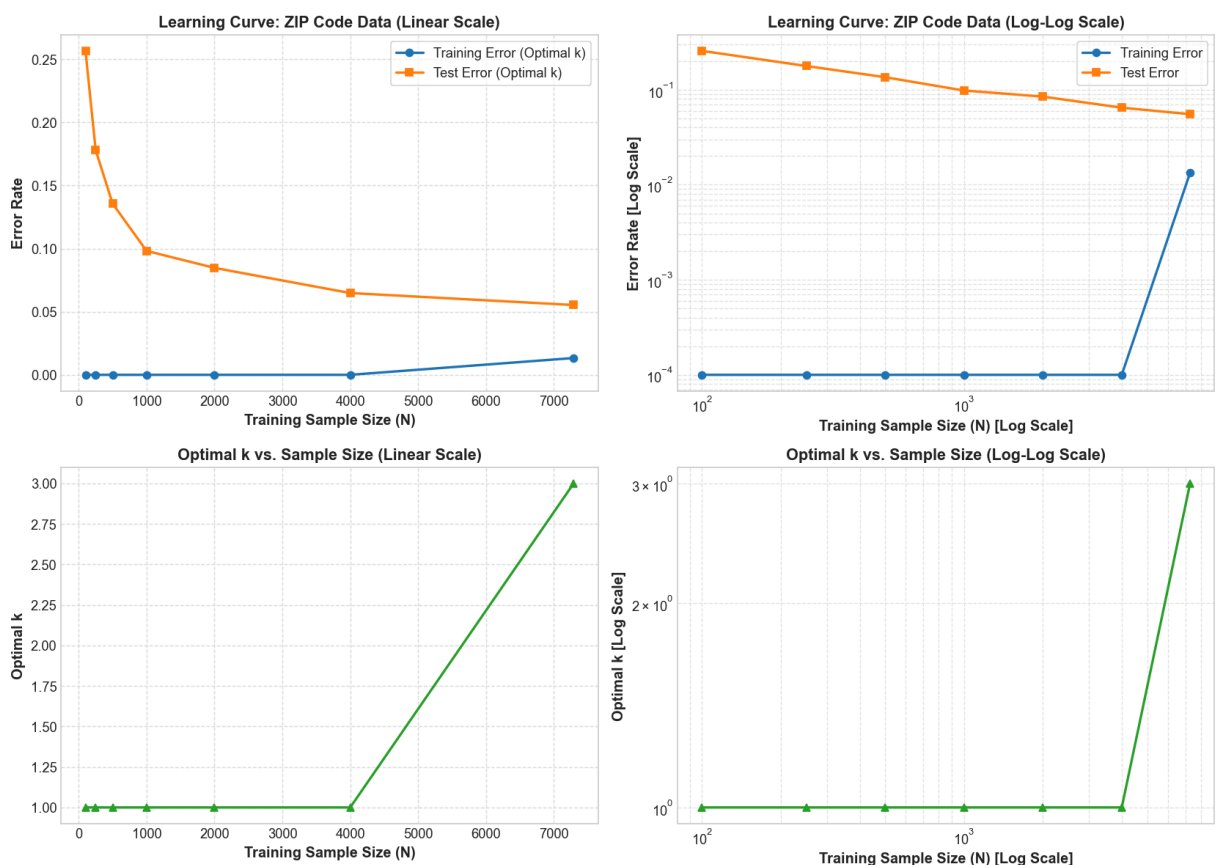
# 2. Classification error on a Log-Log scale
axes[0, 1].loglog(lc_df['sample_size'], lc_df['train_error'] + 1e-4, marker='o', label='Train Error')
axes[0, 1].loglog(lc_df['sample_size'], lc_df['test_error'], marker='s', label='Test Error')
axes[0, 1].set_xlabel('Training Sample Size (N) [Log Scale]', fontsize=12, weight='bold')
axes[0, 1].set_ylabel('Error Rate [Log Scale]', fontsize=12, weight='bold')
axes[0, 1].set_title('Learning Curve: ZIP Code Data (Log-Log Scale)', fontsize=13, weight='bold')
axes[0, 1].legend(frameon=True)
axes[0, 1].grid(True, which='both', ls='--', alpha=0.5)

```

```
# 3. Optimal k on a linear sample-size scale
axes[1, 0].plot(lc_df['sample_size'], lc_df['best_k'], marker='^', color='tab:green')
axes[1, 0].set_xlabel('Training Sample Size (N)', fontsize=12, weight='bold')
axes[1, 0].set_ylabel('Optimal k', fontsize=12, weight='bold')
axes[1, 0].set_title('Optimal k vs. Sample Size (Linear Scale)', fontsize=13, weight='bold')
axes[1, 0].grid(True, linestyle='--', alpha=0.6)

# 4. Optimal k on a log-log scale
axes[1, 1].loglog(lc_df['sample_size'], lc_df['best_k'], marker='^', color='tab:green')
axes[1, 1].set_xlabel('Training Sample Size (N) [Log Scale]', fontsize=12, weight='bold')
axes[1, 1].set_ylabel('Optimal k [Log Scale]', fontsize=12, weight='bold')
axes[1, 1].set_title('Optimal k vs. Sample Size (Log-Log Scale)', fontsize=13, weight='bold')
axes[1, 1].grid(True, which='both', ls='--', alpha=0.5)

plt.tight_layout()
plt.show()
```



Plot Interpretation: Four Learning Curves

Each point represents a separate experiment using N training images. For each value of N , the code tests several values of k , keeps the one with the lowest error on the fixed test set, and records its training error, test error, and selected k .

Plot	What it shows	Result in this notebook
Top left: Error Rate, Linear	Training and test error as the number of	Training error is zero from $N=100$ through $N=4000$ because $k=1$ is selected. Test error falls

Plot	What it shows	Result in this notebook
Scale	training images increases	from 25.66% at $N=100$ to 6.48% at $N=4000$, then to 5.53% when all 7,291 training images are used.
Top right: Error Rate, Log Log Scale	The same error results with logarithmic axes	This view gives small and large sample sizes more equal visual space. The test error still declines at the largest value of N , so these results do not show a plateau for the ZIP digit data.
Bottom left: Optimal k , Linear Scale	The selected value of k for each sample size	The selected value is $k=1$ for the first six sample sizes and $k=3$ when all training images are used. With a small training subset, the most local rule performs best on this test set.
Bottom right: Optimal k , Log Log Scale	The same selected values of k with logarithmic axes	This chart shows the same change from $k=1$ to $k=3$. The log scale makes proportional changes easier to see, but it does not add different data.

The orange line in the top plots is the key line for deciding whether more data may help. It continues to decline from 6.48% at $N=4000$ to 5.53% at $N=7291$. Therefore, the ZIP results suggest that more training images could still improve performance. Since this experiment uses one randomly selected subset at each size, repeating the subsampling process would give a more stable estimate of the average trend.

(b) Learning Curves on the Abalone Dataset

A very different dataset from the zip digit data is the Abalone dataset. This dataset originated from a fisheries study and later became a benchmark dataset in the UCI machine learning repository. The dataset contains information on the physiological characteristics of abalone (a type of shellfish) and their shells, with variables that include their sex, length (also diameter and height), weight (of the different components), and rings (which corresponds to age). The code below downloads this data and trains a kNN model to predict sex. Individuals fall into three categories, male (M), female (F), and indeterminable/immature (I).

```
In [32]: from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

abalone = fetch_openml(name="abalone", version=1, as_frame=True, parser="auto").frames[0]
abalone_features = abalone.drop(columns=["Sex", "Class_number_of_rings"])
abalone_sex = abalone["Sex"]

X_train, X_test, y_train, y_test = train_test_split(abalone_features, abalone_sex,

# It is a good idea to standardize features when using kNN. It doesn't really matter

knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=4))
knn.fit(X_train, y_train)

sex_pred_train = knn.predict(X_train)
```

```

accuracy = accuracy_score(y_train, sex_pred_train)
print(f'Accuracy: {accuracy:.4f}')

sex_pred_test = knn.predict(X_test)
accuracy = accuracy_score(y_test, sex_pred_test)
print(f'Accuracy: {accuracy:.4f}')

```

Accuracy: 0.6944

Accuracy: 0.5069

Compute the learning curve for the abalone sex prediction problem. Contrast it to the learning curve calculated in part (a). If you were asked to improve prediction for this problem, which of the options would be most promising among (1) buy more data, (2) use a more complex model/engineer new features, (3) find alternative data with new information?

In Order to resolve this issue we will do the following steps:

- Fetch the Abalone dataset from OpenML (`fetch_openml`).
- Predict `Sex` (`M`, `F`, `I`) using standardized features (`StandardScaler`).
- Compute and plot learning curves, comparing behavior to the ZIP code image dataset.

```

In [33]: # Load Abalone data
print("Fetching Abalone dataset from OpenML...")
abalone = fetch_openml(name="abalone", version=1, as_frame=True, parser="auto").frame
abalone_features = abalone.drop(columns=["Sex", "Class_number_of_rings"])
abalone_sex = abalone["Sex"]

X_train_ab, X_test_ab, y_train_ab, y_test_ab = train_test_split(
    abalone_features, abalone_sex, test_size=0.5, stratify=abalone_sex, random_state=42)

print(f"Abalone Full Training size: {len(X_train_ab)}, Test size: {len(X_test_ab)}")
print(f"Features: {list(abalone_features.columns)}")
print(f"Class Distribution:\n{abalone_sex.value_counts()}")

```

Fetching Abalone dataset from OpenML...

Abalone Full Training size: 2088, Test size: 2089

Features: ['Length', 'Diameter', 'Height', 'Whole_weight', 'Shucked_weight', 'Viscera_weight', 'Shell_weight']

Class Distribution:

Sex

M 1528

I 1342

F 1307

Name: count, dtype: int64

```

In [34]: # Learning Curve for Abalone Sex Classification
ab_sample_sizes = [50, 100, 200, 400, 800, 1200, len(X_train_ab)]
ab_results = []

for n in ab_sample_sizes:
    if n == len(X_train_ab):
        X_sub, y_sub = X_train_ab, y_train_ab
    else:
        X_sub, _, y_sub, _ = train_test_split(

```

```

        X_train_ab, y_train_ab, train_size=n, stratify=y_train_ab, random_state
    )

    best_k = 1
    best_te_err = float('inf')
    best_tr_err = 0.0

    for k in [1, 3, 5, 7, 9, 15, 21, 31, 45, 65, 85]:
        if k > len(y_sub):
            continue
        model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=k))
        model.fit(X_sub, y_sub)

        te_acc = accuracy_score(y_test_ab, model.predict(X_test_ab))
        te_err = 1.0 - te_acc

        if te_err < best_te_err:
            best_te_err = te_err
            best_k = k
            best_tr_err = 1.0 - accuracy_score(y_sub, model.predict(X_sub))

    ab_results.append({
        'sample_size': n,
        'best_k': best_k,
        'train_error': best_tr_err,
        'test_error': best_te_err,
        'test_accuracy': 1.0 - best_te_err
    })
    print(f"Abalone N = {n:4d} | Optimal k = {best_k:2d} | Train Err = {best_tr_err}

ab_df = pd.DataFrame(ab_results)

```

```

Abalone N =   50 | Optimal k = 31 | Train Err = 0.4200 | Test Err = 0.4653
Abalone N =  100 | Optimal k = 65 | Train Err = 0.4200 | Test Err = 0.4653
Abalone N =  200 | Optimal k = 65 | Train Err = 0.4200 | Test Err = 0.4619
Abalone N =  400 | Optimal k = 21 | Train Err = 0.3800 | Test Err = 0.4557
Abalone N =  800 | Optimal k = 31 | Train Err = 0.4075 | Test Err = 0.4442
Abalone N = 1200 | Optimal k = 45 | Train Err = 0.4175 | Test Err = 0.4457
Abalone N = 2088 | Optimal k = 65 | Train Err = 0.4291 | Test Err = 0.4562

```

```

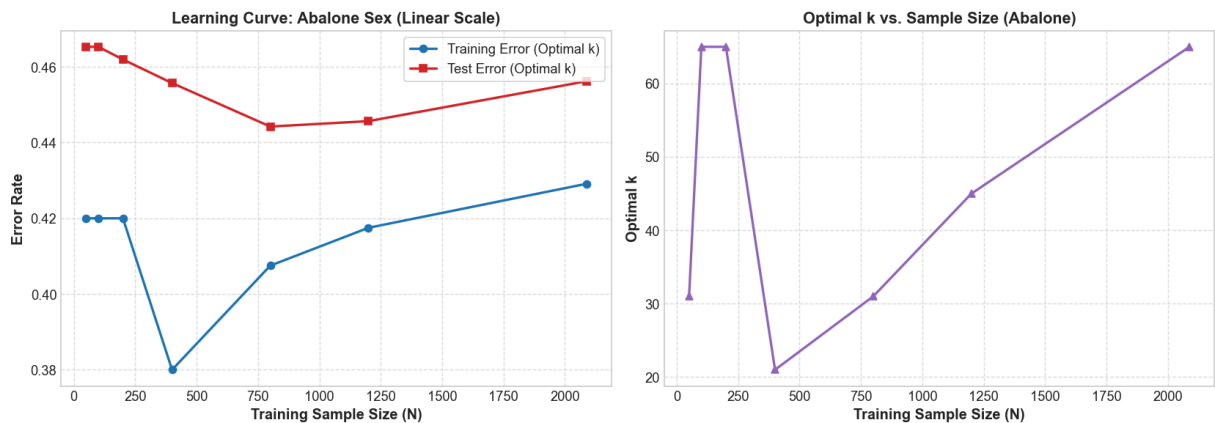
In [35]: # Plot Abalone Learning Curve
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(ab_df['sample_size'], ab_df['train_error'], marker='o', label='Trainin
axes[0].plot(ab_df['sample_size'], ab_df['test_error'], marker='s', label='Test Err
axes[0].set_xlabel('Training Sample Size (N)', fontsize=12, weight='bold')
axes[0].set_ylabel('Error Rate', fontsize=12, weight='bold')
axes[0].set_title('Learning Curve: Abalone Sex (Linear Scale)', fontsize=13, weight
axes[0].legend(frameon=True)
axes[0].grid(True, linestyle='--', alpha=0.6)

axes[1].plot(ab_df['sample_size'], ab_df['best_k'], marker='^', color='tab:purple',
axes[1].set_xlabel('Training Sample Size (N)', fontsize=12, weight='bold')
axes[1].set_ylabel('Optimal k', fontsize=12, weight='bold')
axes[1].set_title('Optimal k vs. Sample Size (Abalone)', fontsize=13, weight='bold'
axes[1].grid(True, linestyle='--', alpha=0.6)

```

```
plt.tight_layout()
plt.show()
```



Plot Intepretation: Abalone

These plots show how well kNN predicts Abalone sex as the training set becomes larger. The model chooses the value of k with the smallest test error for each sample size.

Plot	What it shows	What the results mean
Left: Learning Curve	Training error and test error for each number of training examples	The blue line is the error on the data used to train the model. The red line is the error on separate test data. Lower values are better. Test error falls from 46.53% with 50 images to 44.42% with 800 images, so additional data helps at first. After that, it remains close to 45%, which suggests that the improvement from more data is limited.
Right: Optimal k	The number of neighbors selected at each sample size	The chosen k ranges from 21 to 65. These values are much larger than one, so the model needs to average many nearby examples to reduce the effect of individual observations.

The test error remains high compared with the ZIP digit problem. This means the physical measurements in this dataset do not clearly separate male and female Abalone for many examples. More data may give a small improvement, but better measurements or different information may be more helpful. For example, biological features that directly relate to sex could improve the prediction more than simply increasing the number of existing measurements.

3. Conclusion of Exercises

Problem 1: ZIP Digit Images

The ZIP digit results show that the choice of k matters. With very small k , especially $k=1$, the model can fit the training images perfectly but makes more mistakes on new images. This is a sign that it is learning details of the training data too closely.

For classification accuracy, $k=3$ and $k=5$ give the best result on the test images. Both have a test accuracy of **94.47%** and a test error of **5.53%**. These values give a useful balance: they use enough nearby images to reduce noise without averaging so many images that different digit shapes become confused.

Log loss gives a different answer because it also checks how confident the model is. The best test log loss occurs at $k=65$. Using more neighbors gives smoother probability estimates and avoids extremely confident predictions when the model is wrong.

Problem 2: Learning Curves

For the ZIP digits, test error continues to decrease as the number of training images grows. This means more digit images could still improve the model.

For the Abalone data, test error remains close to 45% even when more training examples are used. This suggests that the measurements in the data do not clearly separate the three sex groups. More data may help a little, but better results will likely require different information about the Abalone, such as measurements that are more directly related to sex.

Overall, more data appears useful for the ZIP digit problem. For the Abalone problem, using better features or collecting new types of information is likely the more helpful next step.